GUÍA RÁPIDA 1 · VISUAL STUDIO 2022

TechStore WPF MVVM paso a paso

Reconstruye desde cero la actividad de ejemplo **ProductApp** (catálogo de productos) con **CommunityToolkit.Mvvm**: modelo, servicio mockup con interfaz, ViewModel con `[ObservableProperty]` / `[RelayCommand]` y la vista XAML enlazada por Binding.

Archivos que vas a crear

- C# **Producto.cs**
  `WpfApp1\Models`
- C# **IProductoService.cs · ProductoService.cs**
  `WpfApp1\Services`
- C# **MainViewModel.cs**
  `WpfApp1\ViewModels`
- XA **MainWindow.xaml · MainWindow.xaml.cs**
  `WpfApp1\Views`
- XA **App.xaml**
  `WpfApp1 (solo se modifica 1 línea)`

Introducción

+ **Crear un proyecto nuevo**
Plantilla «Aplicación WPF», .NET 8 · `Paso 1`

↓ **Instalar CommunityToolkit.Mvvm**
NuGet + carpetas MVVM · `Pasos 2–3`

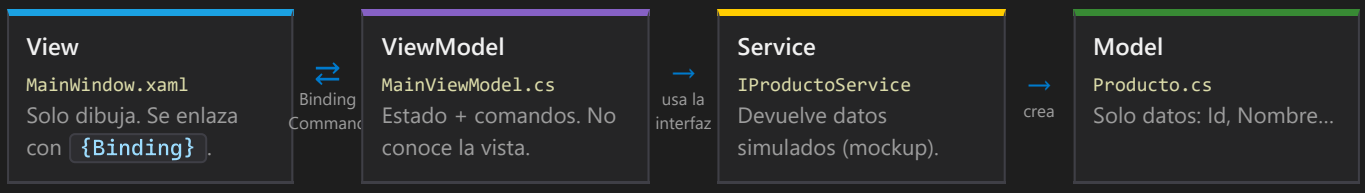
{ } **Model · Service · ViewModel**
La lógica, sin tocar la vista · `Pasos 4–6`

▶ **Vista XAML, DataContext y F5**
Enlazar, ejecutar y probar · `Pasos 7–10`

⏱ **Tiempo estimado: ~35 min**

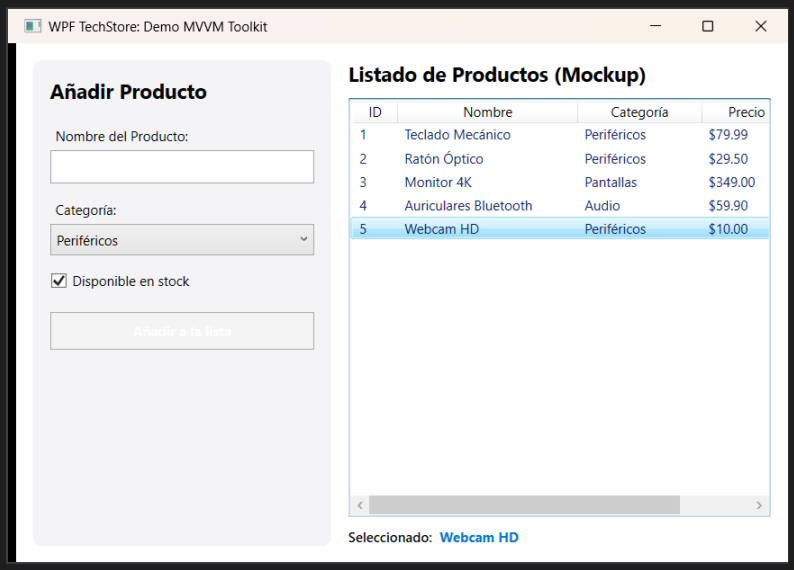


MVVM Cómo encaja todo



El **code-behind** (`MainWindow.xaml.cs`) es quien «pega» las piezas: crea el servicio, se lo pasa al ViewModel por el constructor y lo asigna como `DataContext` .

00 Qué vas a construir



Resultado final tras añadir «Webcam HD» y seleccionarlo.

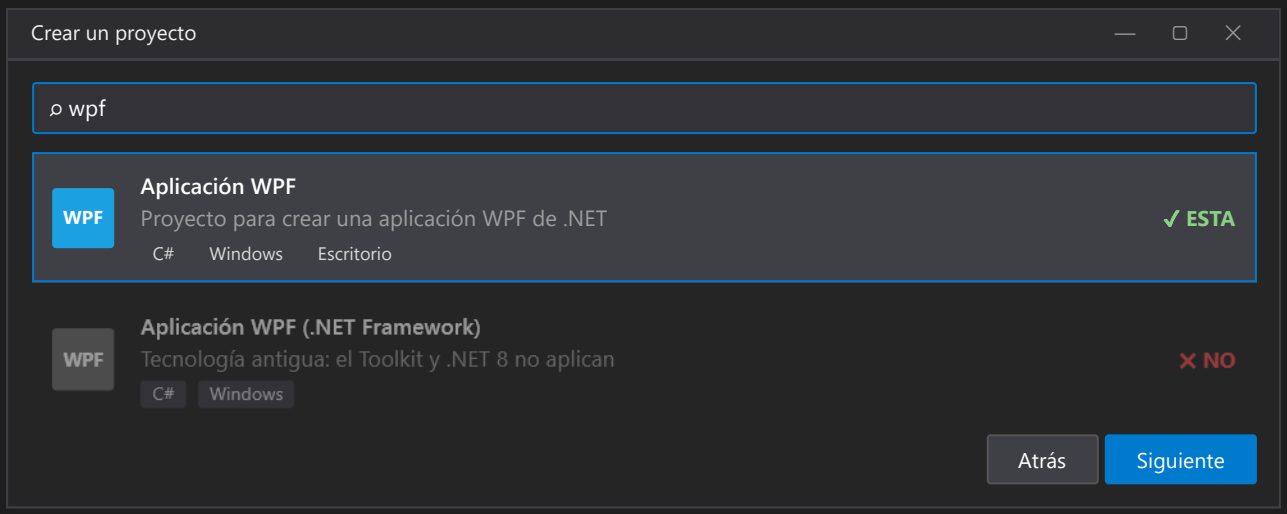
- [ ] Al abrir, el listado carga **4 productos** del servicio mockup.
- [ ] El botón solo se activa si hay **nombre**.
- [ ] «Añadir a la lista» inserta una fila sin refrescar nada a mano.
- [ ] Al seleccionar una fila se muestra su nombre abajo.

Y todo ello **sin un solo evento Click** en el code-behind: solo Binding y Command.

01 Crear el proyecto en Visual Studio

1. Abre **Visual Studio 2022** y en la ventana de inicio pulsa `Crear un proyecto nuevo` (o desde dentro: `Archivo` `Nuevo` `Proyecto...` `Ctrl` + `Mayús` + `N`).

2. En el buscador escribe `WPF`, filtra por **C#** y elige **Aplicación WPF**. ¡Cuidado con la plantilla parecida!



3 **Configurar el nuevo proyecto**

Configurar el nuevo proyecto

Nombre del proyecto: WpfApp1

Ubicación: C:\Users\alumno\Desktop\

Nombre de la solución: WpfApp1

☑ Colocar la solución y el proyecto en el mismo directorio

4 **Información adicional** → Crear

Información adicional

Marco: .NET 8.0 (Compatibilidad a largo plazo)

Si el PC del examen tiene otra versión (.NET 9/10) también funciona: el código es idéntico.

Atrás | Crear

> ⚠ **Llama al proyecto exactamente WpfApp1**
> Todo el código usa los espacios de nombres `WpfApp1.Models`, `WpfApp1.Services`... Si le pones otro nombre, tendrás que cambiar `WpfApp1` en cada archivo.

02 Instalar CommunityToolkit.Mvvm (NuGet)

1 En el **Explorador de soluciones** (`Ctrl` + `Alt` + `L`) haz **clic derecho** sobre el proyecto `WpfApp1`:

Clic dcho. WpfApp1 | Administrar paquetes NuGet...

2 Pestaña **Examinar** → busca `CommunityToolkit.Mvvm` (autor: Microsoft) → **Instalar** → acepta la licencia.

3 **Alternativa rápida** por consola:

Herramientas | NuGet | Consola

NuGet: WpfApp1

Examinar | Instalado | Actualizaciones

CommunityToolkit.Mvvm

.N **CommunityToolkit.Mvvm** by Microsoft
MVVM Toolkit: ObservableObject, RelayCommand...
Instalar

PS Consola del Administrador de paquetes

```
1  PM> Install-Package CommunityToolkit.Mvvm
```

> ✓ **Comprueba que se instaló**
> En el Explorador de soluciones: Dependencias | Paquetes | CommunityToolkit.Mvvm. Si haces doble clic en el proyecto verás en el `.csproj` una línea `<PackageReference Include="CommunityToolkit.Mvvm" … />`.

03 Crear las carpetas MVVM

Repite 4 veces:
Clic dcho. WpfApp1 | Agregar | Nueva carpeta y escribe el nombre:

- **Models** → clases de datos
- **Services** → interfaz + servicio mockup
- **ViewModels** → lógica de presentación
- **Views** → ventanas XAML

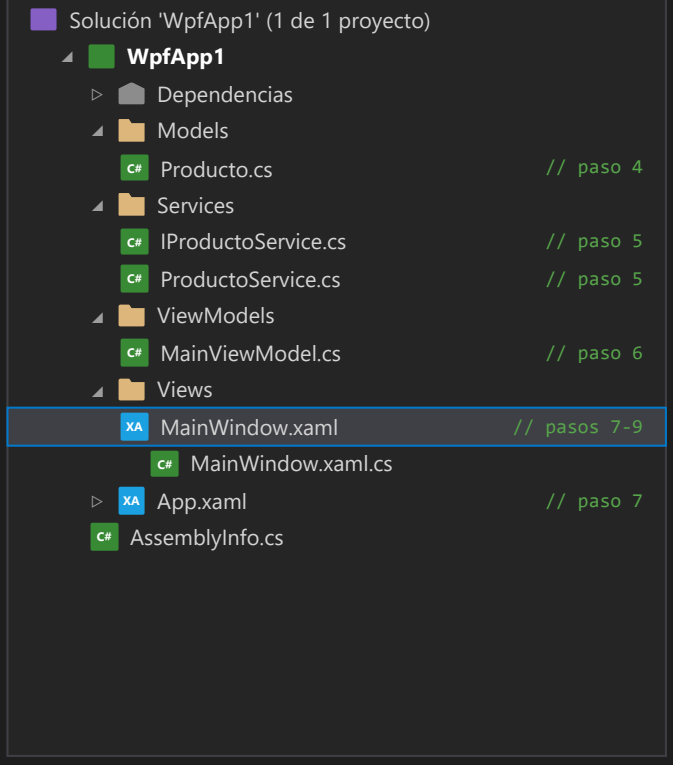
Cómo crear cada archivo

Clase: Clic dcho. carpeta | Agregar | Clase...
`Mayús` + `Alt` + `C`

Interfaz: Agregar | Nuevo elemento... | Interfaz
`Ctrl` + `Mayús` + `A`

> **VS pone el namespace por ti**
> Al crear una clase dentro de `Models`, Visual Studio escribe automáticamente `namespace WpfApp1.Models`. Por eso es importante crear la carpeta antes que el archivo.

Explorador de soluciones — estado final



04 Modelo: Producto

Models/Producto.cs

Clase «tonta»: solo propiedades, sin lógica. Representa **una fila** del listado.

Producto.cs — WpfApp1 › Models › Producto

```
1   namespace WpfApp1.Models
2   {
3       public class Producto
4       {
5           public int Id { get; set; }
6           public string Nombre { get; set; } = string.Empty;
7           public string Categoria { get; set; } = string.Empty;
8           public decimal Precio { get; set; }
9           public bool Disponible { get; set; }
10      }
11  }
```

> `= string.Empty` evita el aviso de nullable (las cadenas nunca empiezan en `null`). `decimal` es el tipo correcto para dinero.

05 Servicio mockup: interfaz + implementación

La **interfaz** es el contrato: «alguien me dará una lista de productos». El ViewModel depende solo de ella, así mañana puedes cambiar el mockup por una base de datos sin tocar el ViewModel.

C# IProductoService.cs — WpfApp1 › Services

```
1   using WpfApp1.Models;
2
3   namespace WpfApp1.Services
4   {
5       public interface IProductoService
6       {
7           List<Producto> ObtenerProductosMockup();
8       }
9   }
```

C# ProductoService.cs — WpfApp1 › Services

```
1   using WpfApp1.Models;
2
3   namespace WpfApp1.Services
4   {
5       public class ProductoService : IProductoService
6       {
7           public List<Producto> ObtenerProductosMockup()
8           {
9               return new List<Producto>
10              {
11                  new Producto { Id = 1, Nombre = "Teclado Mecánico", Categoria = "Periféricos",
12                                 Precio = 79.99m, Disponible = true },
13                  new Producto { Id = 2, Nombre = "Ratón Óptico", Categoria = "Periféricos",
14                                 Precio = 29.50m, Disponible = true },
15                  new Producto { Id = 3, Nombre = "Monitor 4K", Categoria = "Pantallas",
16                                 Precio = 349.00m, Disponible = false },
17                  new Producto { Id = 4, Nombre = "Auriculares Bluetooth", Categoria = "Audio",
18                                 Precio = 59.90m, Disponible = true }
19              };
20          }
21      }
22  }
```

> **¿Por qué no hay `using System.Collections.Generic;`?**
> Los proyectos .NET modernos tienen `ImplicitUsings` activado: `List<T>`, `System`, `System.Linq`… ya están importados. Si los pones, no pasa nada.

06 ViewModel con el Toolkit

ViewModels/MainViewModel.cs

El corazón del ejercicio. Las líneas resaltadas son las «mágicas» del Toolkit: generan código por ti al compilar (Source Generators).

C# MainViewModel.cs — WpfApp1 › ViewModels

```
1   using System.Collections.ObjectModel;
2   using CommunityToolkit.Mvvm.ComponentModel;
3   using CommunityToolkit.Mvvm.Input;
4   using WpfApp1.Models;
5   using WpfApp1.Services;
6
7   namespace WpfApp1.ViewModels
```

```
{
    public partial class MainViewModel : ObservableObject
    {
        private readonly IProductoService _productoService;

        // Lista observable para refrescar el ListView automáticamente
        [ObservableProperty]
        private ObservableCollection<Producto> _productos = new();

        // Categorías disponibles para el ComboBox
        [ObservableProperty]
        private List<string> _categorias = new() { "Periféricos", "Pantallas", "Audio", "Otros" };

        // Campos del formulario para añadir un nuevo producto
        [ObservableProperty]
        [NotifyCanExecuteChangedFor(nameof(AgregarProductoCommand))]
        private string _nuevoNombre = string.Empty;

        [ObservableProperty]
        private string _nuevaCategoria = "Periféricos";

        [ObservableProperty]
        private decimal _nuevoPrecio = 10.00m;

        [ObservableProperty]
        private bool _nuevoDisponible = true;

        // Producto seleccionado en la lista
        [ObservableProperty]
        private Producto? _productoSeleccionado;

        // Inyección de dependencias por constructor
        public MainViewModel(IProductoService productoService)
        {
            _productoService = productoService;
            CargarDatos();
        }

        private void CargarDatos()
        {
            var datos = _productoService.ObtenerProductosMockup();
            Productos = new ObservableCollection<Producto>(datos);
        }

        // Validación: Solo permite agregar si el nombre no está vacío
        private bool CanAgregar => !string.IsNullOrWhiteSpace(NuevoNombre);

        [RelayCommand(CanExecute = nameof(CanAgregar))]
        private void AgregarProducto()
        {
            var nuevo = new Producto
            {
                Id = Productos.Count + 1,
                Nombre = NuevoNombre,
                Categoria = NuevaCategoria,
                Precio = NuevoPrecio,
                Disponible = NuevoDisponible
            };

            Productos.Add(nuevo);

            // Limpiamos el formulario
            NuevoNombre = string.Empty;
        }
    }
}
```

Lo que escribes → lo que genera el Toolkit

Tú escribes		El Toolkit genera / hace
`public partial class … : ObservableObject`	→	Implementa `INotifyPropertyChanged`. **partial** es obligatorio para que el generador añada código.
`[ObservableProperty] private string _nuevoNombre;`	→	Propiedad pública `NuevoNombre` que avisa a la vista al cambiar. Quita el `_` y pone mayúscula.
`[RelayCommand] private void AgregarProducto()`	→	Propiedad `AgregarProductoCommand` (nombre del método + **Command**) para el `Button`.
`CanExecute = nameof(CanAgregar)`	→	Si `CanAgregar` es `false`, el botón se **desactiva solo**.
`[NotifyCanExecuteChangedFor(nameof(AgregarProductoCommand))]`	→	Cada vez que cambia `NuevoNombre`, vuelve a evaluar `CanAgregar` (activa/desactiva el botón al teclear).
`ObservableCollection<Producto>`	→	Al hacer `Productos.Add(…)` el `ListView` se refresca sin hacer nada más.

> ⚠ **Usa la propiedad, no el campo**
> Dentro del ViewModel escribe `NuevoNombre`, nunca `_nuevoNombre`. Si usas el campo, la vista no se entera del cambio y VS marca el aviso `MVVMTK0034`.

07 Mover MainWindow a la carpeta Views

La plantilla crea `MainWindow.xaml` en la raíz. Al moverlo a `Views` hay que actualizar **3 sitios** o no arrancará:

1. **Arrastra** `MainWindow.xaml` en el Explorador de soluciones hasta la carpeta `Views` (su `.xaml.cs` se mueve con él).
2. En `MainWindow.xaml` cambia `x:Class="WpfApp1.MainWindow"` por `x:Class="WpfApp1.Views.MainWindow"`.
3. En `MainWindow.xaml.cs` cambia `namespace WpfApp1` por `namespace WpfApp1.Views`.
4. En `App.xaml` cambia la ventana de inicio:

XAML App.xaml × WpfApp1

```xaml
<Application x:Class="WpfApp1.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="Views/MainWindow.xaml">
    <Application.Resources>
    </Application.Resources>
</Application>
```

Lista de errores 2 Errores | Si te olvidas de un paso...

	Código	Descripción	Causa → solución
✕	CS0103	El nombre 'InitializeComponent' no existe en el contexto actual	`x:Class` y `namespace` no coinciden → pasos 2 y 3.
✕	IOException	Cannot locate resource 'mainwindow.xaml' (al pulsar F5)	`StartupUri` apunta a la ruta vieja → paso 4.

08 Vista XAML

Views/MainWindow.xaml

Un `Grid` de 2 columnas: formulario (280 px) + listado (resto). Abre el archivo, borra el `<Grid>` vacío de la plantilla y escribe esto (mantén tu `x:Class` del paso 7):

XAML MainWindow.xaml × WpfApp1 › Views

```xaml
<Window x:Class="WpfApp1.Views.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="WPF TechStore: Demo MVVM Toolkit" Height="500" Width="700"
        WindowStartupLocation="CenterScreen">

    <Grid Margin="15">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="280"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>

        <!-- PANEL IZQUIERDO: Formulario de alta -->
        <Border Grid.Column="0" Background="#F4F4F6" CornerRadius="8"
                Padding="15" Margin="0,0,15,0">
            <StackPanel>
                <TextBlock Text="Añadir Producto" FontSize="18" FontWeight="Bold"
                           Margin="0,0,0,15"/>

                <!-- Label + TextBox -->
                <Label Content="Nombre del Producto:" />
                <TextBox Text="{Binding NuevoNombre, UpdateSourceTrigger=PropertyChanged}"
                         Padding="6" Margin="0,0,0,10"/>

                <!-- Label + ComboBox -->
                <Label Content="Categoría:" />
                <ComboBox ItemsSource="{Binding Categorias}"
                          SelectedItem="{Binding NuevaCategoria}"
                          Padding="6" Margin="0,0,0,10"/>

                <!-- CheckBox -->
                <CheckBox Content="Disponible en stock"
                          IsChecked="{Binding NuevoDisponible}"
                          Margin="0,5,0,20"/>

                <!-- Button vinculado al Command -->
                <Button Content="Añadir a la lista"
                        Command="{Binding AgregarProductoCommand}"
                        Padding="8" Background="#007ACC" Foreground="White"
                        FontWeight="Bold"/>
            </StackPanel>
        </Border>

        <!-- PANEL DERECHO: Lista de productos -->
        <Grid Grid.Column="1">
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="*"/>
                <RowDefinition Height="Auto"/>
```

```
50              </Grid.RowDefinitions>
51
52              <TextBlock Grid.Row="0" Text="Listado de Productos (Mockup)"
53                         FontSize="18" FontWeight="Bold" Margin="0,0,0,10"/>
54
55              <!-- ListView con GridView -->
56              <ListView Grid.Row="1" ItemsSource="{Binding Productos}"
57                        SelectedItem="{Binding ProductoSeleccionado}">
58                  <ListView.View>
59                      <GridView>
60                          <GridViewColumn Header="ID" Width="40"
61                                          DisplayMemberBinding="{Binding Id}"/>
62                          <GridViewColumn Header="Nombre" Width="160"
63                                          DisplayMemberBinding="{Binding Nombre}"/>
64                          <GridViewColumn Header="Categoría" Width="110"
65                                          DisplayMemberBinding="{Binding Categoria}"/>
66                          <GridViewColumn Header="Precio" Width="80"
67                                          DisplayMemberBinding="{Binding Precio, StringFormat={}{0:C}}"/>
68                          <GridViewColumn Header="Stock" Width="60"
69                                          DisplayMemberBinding="{Binding Disponible}"/>
70                      </GridView>
71                  </ListView.View>
72              </ListView>
73
74              <!-- TextBlock inferior con el elemento seleccionado -->
75              <TextBlock Grid.Row="2" Margin="0,10,0,0" FontWeight="SemiBold">
76                  <Run Text="Seleccionado: " />
77                  <Run Text="{Binding ProductoSeleccionado.Nombre, FallbackValue='Ninguno'}"
78                       Foreground="#007ACC" FontWeight="Bold"/>
79              </TextBlock>
80          </Grid>
81      </Grid>
82  </Window>
```

Mapa de enlaces: control → propiedad del ViewModel

Control XAML	Atributo	Propiedad / comando del VM
`TextBox`	`Text`	`NuevoNombre` con `UpdateSourceTrigger=PropertyChanged` (actualiza al teclear, si no el botón no se activa hasta salir del cuadro)
`ComboBox`	`ItemsSource` / `SelectedItem`	`Categorias` (opciones) / `NuevaCategoria` (elegida)
`CheckBox`	`IsChecked`	`NuevoDisponible`
`Button`	`Command`	`AgregarProductoCommand` (generado por `[RelayCommand]`)
`ListView`	`ItemsSource` / `SelectedItem`	`Productos` / `ProductoSeleccionado`
`GridViewColumn`	`DisplayMemberBinding`	Propiedades de `Producto`: `Id`, `Nombre`... `StringFormat={}{0:C}` = moneda (el `{}` es un escape obligatorio)
`Run`	`Text`	`ProductoSeleccionado.Nombre`; `FallbackValue` muestra «Ninguno» si no hay selección

09 Code-behind: asignar el DataContext

Views/MainWindow.xaml.cs

Abre el `.xaml.cs` (despliega el `.xaml` con ▷ o pulsa `F7` desde el XAML). Aquí se hace la **inyección de dependencias manual**:

MainWindow.xaml.cs — WpfApp1 › Views

```
using System.Windows;
using WpfApp1.Services;
using WpfApp1.ViewModels;

namespace WpfApp1.Views
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            // 1. Instanciamos el servicio mockup
            IProductoService servicioMock = new ProductoService();

            // 2. Asignamos el DataContext inyectando la dependencia
            DataContext = new MainViewModel(servicioMock);
        }
    }
}
```

> **¿Por qué aquí y no en el XAML?**
> El ViewModel necesita un `IProductoService` en su constructor; desde XAML no se le puede pasar. Además el ViewModel no sabe qué servicio concreto usa: eso lo decide quien lo crea.
> El ejemplo original incluye `d:DataContext="{d:DesignInstance Type=vm:MainViewModel}"` en el `<Window>`: solo sirve para que el diseñador sugiera los nombres en los Binding. Es opcional.

10 Compilar, ejecutar y probar

Compila con `Ctrl` + `Mayús` + `B` y ejecuta con `F5` (con depurador) o `Ctrl` + `F5` (sin depurador, más rápido). En la ventana **Salida** debes ver:

Salida — Mostrar salida de: Compilación

```
Compilación iniciada a las 9:02...
1>------ Operación Compilar iniciada: proyecto: WpfApp1, configuración: Debug Any CPU ------
1>WpfApp1 -> C:\Users\alumno\Desktop\WpfApp1\bin\Debug\net8.0-windows\WpfApp1.dll
========== Compilar: 1 correcto, 0 incorrectos, 0 actualizado, 0 omitido ==========
```

Checklist de prueba

- [ ] Al arrancar aparecen los **4 productos** del mockup.
- [ ] Con el nombre vacío el botón está **desactivado**.
- [ ] Al escribir una letra el botón se **activa**.
- [ ] Al pulsarlo aparece una fila nueva (ID 5) y el cuadro se **vacía**.
- [ ] Al hacer clic en una fila, abajo se ve **«Seleccionado: …»**.

> **¿El precio sale en dólares ($79.99)?**
> WPF usa por defecto la cultura en-US. Añade al `<Window>` el atributo `Language="es-ES"` y verás **79,99 €**.

> **Mejora opcional: pedir el precio**
> El ViewModel ya tiene `NuevoPrecio`, pero el XAML no lo usa. Añade bajo el ComboBox:
> `<TextBox Text="{Binding NuevoPrecio}"/>`

11 Errores típicos y cómo arreglarlos

Lista de errores ⊗ Errores ⚠ Advertencias ⓘ Mensajes

	Código	Mensaje	Solución
⊗	CS0246	No se encontró el tipo o el espacio de nombres 'CommunityToolkit'	Falta el paquete NuGet (paso 2).
⊗	CS0260	Falta el modificador parcial en la declaración de tipo 'MainViewModel'	Escribe `public partial class`.
⊗	CS0103	El nombre 'AgregarProductoCommand' no existe	Falta `[RelayCommand]` encima del método o el NuGet.
⊗	CS0103	El nombre 'InitializeComponent' no existe	`x:Class` ≠ `namespace` (paso 7).
⚠	MVVMTK0034	Se hace referencia directa al campo '_nuevoNombre'...	Usa la propiedad `NuevoNombre`.
ⓘ	Binding	La lista/ComboBox sale vacía, sin error	Nombre mal escrito en un `{Binding}` o falta el `DataContext`. Mira Depurar | Ventanas | Errores de enlace XAML.

Atajos de Visual Studio que te ahorran tiempo

Atajo	Qué hace	Atajo	Qué hace
`F5` / `Ctrl`+`F5`	Ejecutar con / sin depurar	`Ctrl`+`.`	Acciones rápidas: añade el `using` que falta
`Ctrl`+`Mayús`+`B`	Compilar solución	`Ctrl`+`K`, `Ctrl`+`D`	Formatear (indentar) el documento
`Mayús`+`Alt`+`C`	Agregar clase	`Ctrl`+`K`, `Ctrl`+`C` / `U`	Comentar / descomentar selección
`F7` / `Mayús`+`F7`	Ir al código / al diseñador XAML	`Ctrl`+`Alt`+`L`	Mostrar Explorador de soluciones
`F12`	Ir a la definición	`Ctrl`+`R`, `Ctrl`+`R`	Renombrar en todo el proyecto

✓ **Resumen en 30 segundos**

Proyecto WPF .NET 8 → NuGet `CommunityToolkit.Mvvm` → carpetas → `Producto` → `IProductoService` + `ProductoService` → `MainViewModel` (`partial`, `[ObservableProperty]`, `[RelayCommand]`) → mover `MainWindow` a `Views` (x:Class + namespace + StartupUri) → XAML con Binding → `DataContext = new MainViewModel(new ProductoService())` → `F5`.